Architecture des Ordinateurs : Représentation des Entiers et Arithmétique en Complément à Deux

Thème principal : Codage en mémoire, représentation des nombres entiers signés/non signés, conversion en complément à deux et détection des dépassements de capacité (Overflow)

Synthèse exécutive : Ce cours aborde la manière dont les ordinateurs codent et manipulent physiquement les entiers en mémoire. Face à l'impossibilité pour un composant électronique de stocker directement un signe '+' ou '-', l'architecture informatique utilise la convention du complément à deux sur un nombre fixe de bits. Ce système unifie l'addition et la soustraction au niveau de l'Unité Arithmétique et Logique (ALU) tout en éliminant le problème du double zéro. Nous détaillons la méthode de conversion pas à pas, les opérations arithmétiques bas niveau, ainsi que la détection des erreurs de dépassement de capacité (overflow), piège classique lors des contrôles et des développements bas niveau.

Problématique matérielle : Pourquoi le Complément à Deux ?

Au niveau matériel, la mémoire informatique et les registres du processeur sont constitués de bascules électroniques ne pouvant stocker que deux états : 0 (absence de tension) ou 1 (présence de tension). Il n'existe aucun composant pour représenter directement le symbole '+'' ou '-'. Pour coder des entiers négatifs, plusieurs conventions ont été inventées. La plus intuitive, nommée 'Signe et Magnitude' (où le bit le plus à gauche indique le signe et les autres la valeur absolue), présente deux défauts majeurs : elle génère deux codages distincts pour zéro (+0 = 0000_2 et -0 = 1000_2 sur 4 bits), ce qui gaspille une combinaison et ralentit les comparaisons, et elle impose de construire des circuits électroniques différents pour l'addition et la soustraction. Le complément à deux résout intégralement ces problèmes : le zéro est unique (0000...0) et la soustraction A - B s'effectue exactement comme l'addition A + (-B) sur le même circuit additionneur de l'ALU.

- Absence de signe matériel : Seuls les niveaux logiques 0 et 1 existent physiquement.
- Inconvénients du 'Signe + Magnitude' : Existence d'un double zéro (+0 et -0) et complexité des circuits.
- Unification ALU : La soustraction devient une simple addition A + (-B).

Cas concret enseignant : *Le professeur utilise la métaphore du compteur kilométrique d'une voiture ancienne à 4 chiffres : si le compteur affiche 0000 km et que vous faites marche arrière d'un kilomètre, le compteur passe à 9999. En binaire sur 8 bits, faire -1 à partir de 00000000 donne 11111111. C'est ce principe qui définit -1 en complément à deux.*

Algorithme de Conversion et Plage de Valeurs

Pour représenter un entier négatif -X en complément à deux sur N bits, l'algorithme comporte trois étapes strictes : 1. Représenter la valeur absolue X en binaire naturel sur N bits (en complétant impérativement par des 0 à gauche jusqu'à atteindre N bits). 2. Inverser chaque bit (transformer les 0 en 1 et les 1 en 0) : c'est le complément à un. 3. Ajouter 1 au résultat binaire obtenu. Exemple sur 8 bits pour -12 : +12 s'écrit 00001100_2. L'inversion donne 11110011_2. En ajoutant 1, on obtient 11110100_2. Le bit de poids fort (MSB) vaut 1, attestant que le nombre est négatif. La plage sur N bits n'est pas symétrique : sur 8 bits, elle va de -128 à +127 car le zéro occupe une case de la partie positive.

- Bit de poids fort (MSB - Most Significant Bit) : Représente le signe (0 = positif, 1 = négatif).
- Plage de valeurs sur N bits : De -2^(N-1) à +(2^(N-1) - 1).
- Méthode des 3 étapes : Valeur absolue, Inversion (Complément à 1), +1.

Cas concret enseignant : *Attention absolue en contrôle : Le professeur rappelle qu'il faut D'ABORD fixer la taille N bits (ex: 8 bits) avant d'inverser. Si vous inverser '1100' sans écrire les zéro de gauche '00001100', vous obtiendrez un résultat faux !*

Soustraction Binaire et Détection du Dépassement de Capacité (Overflow)

Lors de l'exécution d'une addition en complément à deux, le processeur travaille sur une taille fixe de registres. Si le résultat d'une opération dépasse la valeur maximale représentable (ou est inférieur à la valeur minimale), un dépassement de capacité (Overflow) se produit, faussant le calcul. Il ne faut pas confondre la retenue sortante (Carry out), qui apparaît sur le bit le plus à gauche et est ignorée en arithmétique signée modulo 2^N, et le bit d'Overflow (OV). La règle fondamentale de détection d'Overflow en examen est la suivante : un Overflow se produit si et seulement si l'on additionne deux opérandes de même signe (deux positifs ou deux négatifs) et que le résultat possède un signe opposé. Si on additionne un nombre positif et un nombre négatif, un Overflow est mathématiquement impossible.

- Retenue sortante (Carry out) vs Dépassement (Overflow).
- Condition d'Overflow : Addition de deux nombres de même signe produisant un résultat de signe opposé.
- Impossibilité d'Overflow : L'addition de deux nombres de signes différents ne peut jamais produire d'Overflow.

Cas concret enseignant : *Le professeur cite l'exemple historique du bogue du processeur d'Ariane 5 (vol 501 en 1996) : un nombre flottant 64 bits a été converti en entier signé 16 bits sans protection contre le dépassement de capacité, provoquant l'autodestruction de la fusée.*

■ Questions & Interventions des Étudiants en Classe

Question étudiant : Pourquoi la retenue tout à gauche (Carry) est-elle jetée lors de l'addition en complément à deux ?
Réponse du professeur : La retenue sortante du bit MSB est abandonnée car l'ALU travaille en arithmétique modulaire (modulo 2^N). Sur un registre fixe de N bits, cette retenue dépasse la capacité physique du registre. Pour des nombres signés, son apparition est normale et ne signifie pas qu'il y a une erreur ; seule la vérification du drapeau d'Overflow (OV) indique une erreur d'extension arithmétique.

Question étudiant : Comment l'ordinateur fait-il la différence entre le nombre binaire 11111111 qui vaut 255 en non-signé et -1 en complément à deux ?
Réponse du professeur : L'ordinateur ne fait pas la différence au niveau de la mémoire : ce ne sont que des bits. C'est le programmeur via le type de variable choisie (ex: 'unsigned char' vs 'signed char' en C) et les instructions d'assemblage utilisées (ex: ADD vs ADDU) qui imposent à l'ALU comment interpréter le résultat et quels drapeaux de contrôle vérifier.

■ Exercice & Démonstration d'Application

Énoncé : Sur une architecture 8 bits, effectuer la soustraction signée : 35 - 75. Détailler l'écriture binaire des opérandes, effectuer l'addition binaire correspondante, donner le résultat décimal et indiquer s'il y a eu un Overflow.

Résolution pas à pas :
1. Conversion de +35 sur 8 bits : 35 = 32 + 2 + 1 = 00100011_2. 2. Conversion de -75 en complément à deux sur 8 bits : - Valeur absolue +75 en binaire sur 8 bits : 64 + 8 + 2 + 1 = 0000100111... -> 01001011_2. - Inversion des bits (Complément à 1) : 10110100_2. - Ajout de 1 (Complément à 2) : 10110100 + 1 = 10110101_2 (Représentation de -75). 3. Pose de l'addition binaire (+35) + (-75) : 00100011 (+35) + 10110101 (-75) ---------- 11011000_2 4. Interprétation du résultat : Le bit MSB est 1, le résultat est négatif. Pour trouver sa valeur absolue, on réapplique le complément à deux sur 11011000_2 : - Inversion : 00100111_2 - Ajout de 1 : 00101000_2 = 32 + 8 = 40 décimal. Le résultat binaire 11011000_2 correspond donc à -40 en décimal. 5. Analyse de l'Overflow : On a additionné un nombre positif (+35) et un nombre négatif (-75). L'addition de deux nombres de signes opposés ne peut jamais produire d'Overflow. Le résultat est exact.

Résultat : **Résultat binaire : 11011000_2 (-40 décimal). Overflow = 0 (Pas de dépassement de capacité). Le calcul 35 - 75 = -40 est correct.**

■ Auto-Évaluation Flash d'Examen

Q : Quelle est la plage des entiers signés représentables sur un mot de 16 bits en complément à deux ?

R : De -32 768 à +32 767 (soit de -2^15 à +2^15 - 1).

Q : Comment détecte-t-on de façon certaine un Overflow lors de l'addition de deux entiers signés ?

R : Un Overflow survient lorsque l'addition de deux nombres de même signe (positif+positif ou négatif+négatif) produit un résultat de signe opposé.

Q : Quel est le résultat de l'inversion et de l'ajout de 1 appliqué au nombre zéro (00000000_2) sur 8 bits ?

R : Inversion = 11111111_2. Ajout de 1 = 100000000_2. Sur 8 bits, la retenue sortante est rejetée et le résultat reste 00000000_2, ce qui prouve l'unicité du zéro.

Fiche académique générée et vérifiée par Kai pour Matéi • Hébergée sur http://88.96.47.180:8765